SMEAN AI · Speech Technology Research

Neural Text-to-Speech for Khmer

VoxCPM2 and Higgs TTS 3 — architecture, training, data requirements, and evaluation

A literature review of the two open-weight multilingual TTS systems that produce intelligible Khmer, prepared as the technical basis for a Khmer speech-synthesis capability at Smean AI.

Document	Literature review — TTS model survey
Subjects	VoxCPM2 (OpenBMB) · Higgs TTS 3 (Boson AI)
Scope	Architecture · training procedure · data requirements · evaluation metrics
Evidence base	Model checkpoints, shipped source code, vendor documentation, and a 400-clip in-house synthesis run
Date	6 September 2026
Status	Internal research document

1. Executive summary

Khmer is a low-resource language for speech synthesis. It has no eSpeak-NG voice, hence no off-the-shelf grapheme-to-phoneme frontend; its script is an abugida with no inter-word spacing, so even tokenising a sentence into words requires a dedicated tool; and no large, clean, permissively-licensed Khmer TTS corpus exists in public. Any Khmer TTS effort therefore starts from a multilingual foundation model rather than from scratch.

Four candidate open-weight models were synthesised over a fixed 100-sentence Khmer test set (50 pure Khmer, 50 code-switched Khmer–English; 400 clips, zero failures) and judged by a native Khmer speaker. Two survived:

- **VoxCPM2** (OpenBMB, 2B parameters, Apache-2.0) — a tokenizer-free diffusion-autoregressive model that predicts continuous acoustic latents rather than discrete codec tokens. Khmer is one of its 30 documented languages, with a self-reported 2.05% CER.
- **Higgs TTS 3** (Boson AI, 4B parameters, research/non-commercial) — a neural-codec language model on a Qwen3-4B backbone with a semantically-grounded audio tokenizer. Khmer is undocumented, yet works.

Two models were eliminated: Fish Audio S2 on correctness (it truncates sentences), and Meta MMS-TTS on prosody (intelligible but flat and robotic).

The four findings that matter

Architecture is the differentiator, not scale. VoxCPM2's tokenizer-free design has no discrete audio vocabulary that can be under-fitted for a low-resource language — the exact failure that makes Fish Audio S2 unusable in Khmer. Higgs TTS 3 is a codec model, but its tokenizer fuses a semantic (HuBERT) branch with an acoustic (DAC) branch, which is the most plausible explanation for Khmer working at all in a language it was never documented to support.

Only one of the two can be trained today. VoxCPM2 ships an official fine-tuning guide, LoRA configs and a dataset validator; a useful Khmer adaptation is a YAML file and a rented 24 GB GPU. Boson ship no training code for Higgs TTS 3 at all — its shipped model class has no loss-returning forward pass — so training it means writing the trainer first.

CER now works for Khmer, and it favours VoxCPM2 decisively. Whisper-large-v3 cannot transcribe Khmer and made the metric meaningless in the first pass. Re-scored with a Khmer-capable ASR, median CER is 2.47% for VoxCPM2 against 8.28% for Higgs TTS 3, 25.12% for MMS and 78.01% for Fish S2-Pro — and the scorer independently reproduces two of OpenBMB's published benchmark figures. VoxCPM2 records the lower CER on 73 of the 100 sentences.

UTMOS is inverted for Khmer, which is why it disagreed with the listener. Across 400 clips the rank correlation between UTMOS and CER is +0.55: the clips it scores highest are the ones that get the Khmer most wrong. Level and silence differences were tested and ruled out as the cause. Naturalness still needs human listening; intelligibility no longer does. Section 6 documents both passes.

The recommendation that follows is set out in Section 7: build on VoxCPM2, keep Higgs TTS 3 as the zero-shot baseline any fine-tune must beat, and spend the remaining evaluation effort on a blind multi-listener test — the one question the numbers still cannot answer is which of the two sounds more natural to Khmer ears.

2. Background: how a modern TTS model is put together

Every zero-shot neural TTS system in current use decomposes into three parts. Naming the parts makes the difference between the two subject models legible.

Stage	Job	Typical realisation
Text frontend	Turn written text into a sequence the model can condition on	A byte/BPE tokenizer, or a G2P phonemizer plus lexicon
Acoustic model	Predict a compact intermediate representation of the speech	An autoregressive transformer, a diffusion model, or both
Vocoder / decoder	Turn that representation into a waveform	A neural codec decoder, a GAN vocoder, or a VAE decoder

2.1 The representation choice: discrete tokens vs continuous latents

The single most consequential design decision is what the acoustic model actually predicts. There are two families, and the two subject models sit on opposite sides.

	Neural-codec LM (discrete)	Continuous-latent (tokenizer-free)
Representation	Speech quantised into integer tokens from a learned codebook, predicted exactly like text tokens	Continuous real-valued vectors, predicted by regression or diffusion
Strength	Stable to train; reuses the whole LLM toolchain; fast, streamable decoding	No quantisation loss; no fixed audio vocabulary to under-fit
Weakness	The codebook is a bottleneck. If it never learned to represent a language's phonation, the LM cannot recover it	Autoregressive continuous prediction accumulates error over long sequences unless carefully constrained
Examples here	**Higgs TTS 3**, Fish Audio S2	**VoxCPM2**

> **Why this matters specifically for Khmer**
>
> A codebook is trained on whatever speech the codec saw. For a language with a small share of the pretraining corpus, the codebook may simply lack codes that represent its phonation faithfully — and no amount of LM capacity fixes that, because the LM can only choose among codes that exist. This is the most credible account of why Fish Audio S2-Pro scores 75.15% CER on Khmer while VoxCPM2 scores 2.05% on the same benchmark. Higgs TTS 3 mitigates it by grounding its codebook semantically rather than purely acoustically.

2.2 Why the Khmer text frontend is the usual blocker — and why it isn't here

Conventional TTS pipelines need a phonemiser. Khmer defeats the standard ones: eSpeak-NG has no Khmer voice, so there is no off-the-shelf G2P; the script stacks subscript consonants and places vowel signs on all four sides of a base glyph; and words are not separated by spaces, so segmentation needs a dedicated tool such as khmertagger.

Both subject models are trained end-to-end on raw text — VoxCPM2 on bytes through a 73,448-entry tokenizer, Higgs TTS 3 through the Qwen3 tokenizer. Neither needs a Khmer phonemiser, lexicon, or word segmenter. That single property removes what is normally the largest engineering block in a low-resource TTS project, and it is why both models can also switch between Khmer and Latin script mid-sentence without being told.

3. VoxCPM2

Property	Value
Organisation	OpenBMB
Parameters	2B (~8 GB VRAM to run)
Family	Tokenizer-free diffusion-autoregressive
Backbone	MiniCPM-4
Audio	16 kHz in (reference/training) → **48 kHz out**
Languages	30 documented, **including Khmer**, plus 9 Chinese dialects
Licence	**Apache-2.0** — commercial use permitted, no attribution required
Weights	huggingface.co/openbmb/VoxCPM2
Paper	Zhou et al., "VoxCPM: Tokenizer-Free TTS for Context-Aware Speech Generation and True-to-Life Voice Cloning", arXiv:2509.24650
Fine-tuning	Official guide, LoRA configs, and a dataset validator

Table 1 — VoxCPM2 at a glance. Architecture figures throughout §3 are read from the checkpoint's own config.json and the voxcpm 2.0.3 package source, not from marketing copy.

3.1 Architecture

A 2B-parameter language model predicts one continuous latent vector per four-frame patch of audio; a small diffusion transformer turns each predicted latent into acoustic features; a separately-trained variational autoencoder decodes those features to a waveform. Because the VAE takes 16 kHz features in and emits 48 kHz audio, super-resolution is built into the decoder rather than bolted on afterwards.

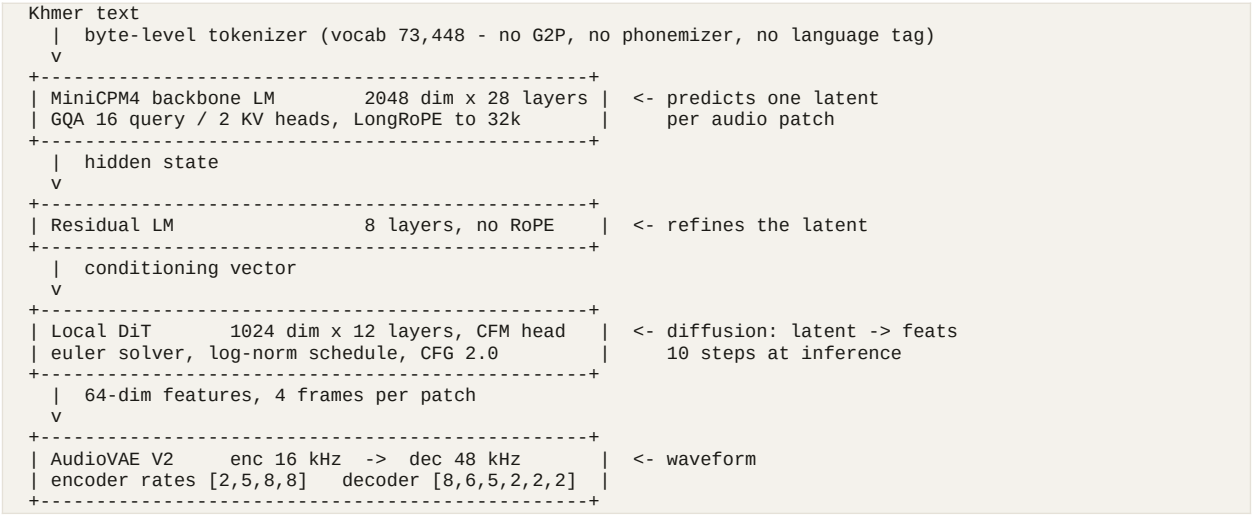

Figure 1 — the VoxCPM2 inference stack. A Local Encoder (1024 dim × 12 layers) sits on the input side, encoding reference audio for voice cloning into the same latent space the LM operates in.

The paper names the stages LocEnc → TSLM → RALM → LocDiT. The TSLM (Text-Semantic Language Model) plans semantic content and prosody through a differentiable quantisation bottleneck rather than hard discrete tokens; the RALM (Residual Acoustic LM) recovers the fine acoustic detail that coarse plan omits; the LocDiT decodes the combination under a diffusion objective. The design is explicitly framed as avoiding the discrete-vs-continuous tradeoff described in §2.1 by adopting a semi-discrete residual representation.

3.2 The numbers that govern training

Property	Value	Why it matters
patch_size	4	One LM step covers 4 VAE frames. Sequence length ≈ chars + duration×25/4.
feat_dim	64	Latent width the diffusion transformer predicts.
AudioVAE frame rate	25 fps	One second of audio ≈ 25 frames ≈ 6.25 LM positions.
max_length	8192	Training sequence cap; longer samples are dropped by the packer.
Encoder sample rate	**16 kHz**	**Training audio must be 16 kHz** — the validator hard-fails on a mismatch.
Output sample rate	48 kHz	Free super-resolution; you do not supply 48 kHz data.
scalar_quantization_latent_dim	512	Scalar quantisation on the latent, not a VQ codebook — this is the “tokenizer-free” part.
inference_cfg_rate	2.0	Classifier-free guidance default, exposed as --cfg-value.

3.3 Where Khmer already stands

Khmer (km) is one of VoxCPM2's 30 documented languages, confirmed in the checkpoint's own model-card metadata. OpenBMB report 2.05% CER for Khmer on their internal 30-language benchmark — the strongest published Khmer figure of any model surveyed. That is close to their own 30-language average of 1.68%, and it should be read as a vendor's upper bound rather than an independently reproduced result.

> **The single most important planning fact in this document**
>
> **You are not adding Khmer to VoxCPM2. You are improving Khmer that is already there.**
>
> The official guide's headline requirement — 500+ hours of target-language data — applies to languages the model does not know. For a language already in the training mix, the task is speaker adaptation, domain adaptation or prosody correction, and the vendor's own recommendations for those are two to three orders of magnitude smaller: tens to hundreds of clips, not hundreds of hours.

3.4 What training data VoxCPM2 needs

Pretraining corpus (for reference)

2M+ hours of multilingual speech across the 30 languages and 9 Chinese dialects. OpenBMB publish no per-language hour breakdown — the same transparency gap every foundation-model vendor in this space has. The Khmer benchmark number is the only available evidence that Khmer received a non-trivial share.

Fine-tuning manifest

Training data is a JSONL file, one JSON object per line:

```
{"audio": "clips/km_0001.wav", "text": "<Khmer transcript>"}
{"audio": "clips/km_0002.wav", "text": "<Khmer transcript>", "ref_audio": "clips/km_0001.wav"}
```

Field	Required	Notes
audio	**yes**	Path to a WAV. Relative paths resolve against the manifest's directory.
text	**yes**	Exact transcript. Raw Khmer — no phonemes, no segmentation, no romanisation.
ref_audio	no	A different clip from the same speaker. Trains the voice-cloning path.
duration	no	Seconds. Lets the length filter skip opening every file — worth it above a few thousand rows.
dataset_id	no	Integer tag for mixing corpora; lets the model condition on source.

The package ships a validator; run it before spending GPU hours. It reports total hours, duration range and text-length distribution, and hard-fails on missing files, sample-rate mismatches, empty transcripts and malformed JSON.

```
voxcpm validate --manifest data/train.jsonl --sample-rate 16000 --verbose
```

Audio requirements

Requirement	Value	Enforcement
Sample rate	**16 kHz**	Hard failure in `validate` on a mismatch.
Format	WAV	Recommended; anything soundfile reads will load.
Clip duration	**3–30 s**	Over 30 s warns (OOM risk); under 0.3 s warns as too short.
Trailing silence	**< 0.5 s**	Not cosmetic — see the callout below.
Loudness	Normalised	Consistent level across the corpus.

> **Trailing silence is the most-cited cause of failure in the official FAQ**

> Clips that end with a long silent tail teach the model that utterances do not end, and it learns to keep generating — producing exactly the runaway output that made Fish Audio S2 unusable in our evaluation. Trim aggressively. This is the highest-value preprocessing step available, and it costs nothing.

How much data

Goal	Method	Data	Realistic for Khmer?
One specific Khmer voice	LoRA r=32	5–50 clips	**Yes** — an afternoon of recording.
Better Khmer prosody, or a domain (news, IVR, education)	LoRA r=32–64	50–500 clips	**Yes — the highest-value target.**
Large-scale Khmer customisation	Full fine-tune	1000+ clips	Plausible with OpenSLR SLR42.
Adding a language from scratch	Full fine-tune, lr 1e-5	**500+ hours**	Not applicable — Khmer is already in.

3.5 Fine-tuning procedure

Environment: Python 3.10–3.11, PyTorch ≥ 2.5.0, CUDA ≥ 12.0, plus tensorboardX, argbind, transformers and librosa. VRAM, at batch_size=16 and max_batch_tokens=8192:

Mode	VRAM	Note
LoRA	~20 GB	A single 24 GB card (A10G / 3090 / 4090) is the pragmatic target.
Full fine-tune	~40 GB	A 48 GB A6000 class card.
Multi-GPU	+~10 GB per card	Gradient buckets and NCCL buffers — not free.

> **Hardware reality check**
>
> The in-house evaluation ran on a 12 GB RTX 3060 — **below the LoRA minimum**. Reaching ~20 GB means dropping batch_size and max_batch_tokens hard and raising grad_accum_steps to compensate, and even then 12 GB is marginal. Renting a single 24 GB card for LoRA is the realistic path; budget for it rather than trying to fit locally.

The LoRA configuration (conf/voxcpm_v2/voxcpm_finetune_lora.yaml), abbreviated to the fields that carry decisions:

```
pretrained_path: /path/to/VoxCPM2/
train_manifest:  /path/to/train.jsonl
val_manifest:    /path/to/val.jsonl

sample_rate:      16000      # must match your audio
out_sample_rate:  48000
batch_size:       16
grad_accum_steps: 1
num_iters:        1000
valid_interval:   500
save_interval:    500

learning_rate:    0.0001     # 1e-4 for LoRA; 1e-5 for a full fine-tune
weight_decay:     0.01
```

```
warmup_steps:     100
max_batch_tokens: 8192

lambdas:
  loss/diff: 1.0            # diffusion loss on the latent
  loss/stop: 1.0            # end-of-utterance prediction

lora:
  enable_lm:   true         # adapt the MiniCPM4 backbone
  enable_dit:  true         # adapt the diffusion transformer
  enable_proj: false
  r:     32
  alpha: 32
  dropout: 0.0
```

- **Choosing the rank.** r=32 clones a speaker; r=64 is for style or language work — which is what Khmer prosody correction is. Set alpha to r or 2r.
- **Keep both adapters on.** The backbone carries linguistic behaviour, the DiT carries acoustic detail, and Khmer adaptation wants both. enable_proj stays false.
- **For a full fine-tune,** delete the lora: block entirely and drop the learning rate tenfold to 1e-5.

```
# single GPU
python scripts/train_voxcpm_finetune.py --config_path conf/voxcpm_v2/voxcpm_finetune_lora.yaml

# multi-GPU
CUDA_VISIBLE_DEVICES=0,1,2,3 torchrun --nproc_per_node=4 \
    scripts/train_voxcpm_finetune.py --config_path conf/voxcpm_v2/voxcpm_finetune_lora.yaml
```

Monitor with TensorBoard: loss/diff should fall steadily, loss/stop should stabilise early, and grad_norm and lr should behave. But select checkpoints by ear, not by loss — the FAQ is explicit that on small datasets the model begins ignoring the text input within a few hundred steps. Checkpoint every 500 steps and listen to each one; the best checkpoint is frequently not the last.

LoRA adapters hot-swap at runtime, so one base model can serve several Khmer voices from a single set of 2B weights:

```
from voxcpm import VoxCPM

model = VoxCPM.from_pretrained(
    "openbmb/VoxCPM2",
    lora_weights_path="/path/to/checkpoints/lora/latest",
)
wav = model.generate(text="<Khmer text>")
```

3.6 Failure modes

Symptom	Cause	Fix
Output ignores the text; reproduces training clips	Overfitting — the classic small-dataset failure, and it arrives fast	Fewer steps, lower LR, more data, lower LoRA rank. Checkpoint often.
Generation runs away, or long trailing noise	**Trailing silence > 0.5 s in the training clips**	Re-trim the corpus. The most common cause per the official FAQ.
Loss will not converge	Bad transcripts, wrong sample rate, misaligned audio	Re-run `voxcpm validate`; spot-check alignment by hand.
Out of memory	Batch too large	Lower batch_size / max_batch_tokens, raise grad_accum_steps.

Khmer improved, English and Chinese degraded	Catastrophic forgetting	Mix Chinese/English data in — or use LoRA and disable it for other languages.

That last row is a genuine argument for LoRA on this project: because the adapter is separable, a Khmer LoRA cannot damage the base model's other 29 languages. Toggle it off and the original model is back, bit for bit. A full fine-tune has no such escape hatch.

4. Higgs TTS 3

Property	Value
Organisation	Boson AI
Parameters	4B
Family	Neural-codec language model
Backbone	Qwen3-4B — 36 layers, hidden 2560, GQA 32 q / 8 kv, head dim 128
Audio	8 codebooks × 1026 vocab at 25 fps → **24 kHz** waveform
Languages	102 claimed (85 “polished”, 17 “usable but less polished”). **Khmer is in neither list.**
Licence	**Boson Higgs TTS 3 Research and Non-Commercial License** — commercial use requires separate negotiation
Weights	huggingface.co/bosonai/higgs-tts-3-4b
Fine-tuning	**None shipped** — no trainer, no dataset loader, no configs

Table 2 — Higgs TTS 3 at a glance. Architecture is read from the checkpoint's config.json, the higgs-audio-v2-tokenizer config, and the modeling_higgs_multimodal_qwen3.py remote code shipped with the weights.

4.1 Two things to establish before anything else

1 — Khmer is an emergent, unsupported capability

Boson report single-digit WER/CER on 102 languages across two tiers. Khmer appears in neither. It is not a low-tier language for this model; it is an unlisted one. Yet it demonstrably produces intelligible Khmer — confirmed across 100 sentences in our own run and audible on Boson's own demo space.

That makes Khmer **real, but unmeasured, unsupported, and carrying no vendor commitment**. If a future release silently drops it, nothing was promised. Everything else in this section rests on that footing.

2 — The licence is not open in the sense VoxCPM2 is

Higgs TTS 3 ships under the Boson Higgs TTS 3 Research and Non-Commercial License. Production use, hosted APIs, embedding it in a product or service, and reselling it all require a separate commercial licence from Boson. A **Creator Use Grant** permits free use — including monetised podcasts, videos and social posts — provided “Boson AI's Higgs Audio” is credited in the audio or prominently in accompanying text.

The practical fork: if the deliverable is a product, VoxCPM2 is the only one of the two shippable without a commercial negotiation. This is a licensing decision, not a quality one, and it is worth settling before any training effort is spent.

4.2 Architecture

```
Khmer text
  |  Qwen3 tokenizer (vocab 151,936) + Higgs control tokens
  v
+-----------------------------------------------------+
```

```
| Qwen3-4B backbone                                    |
| 36 layers . hidden 2560 . GQA 32 q / 8 kv . head 128  |
| RoPE theta=1e6 . 32,768 max pos . 8,192 train seq len |
| tied text embeddings                                 |
+------------------------------------------------------+
   ^ fused multi-codebook embedding    v fused multi-codebook head
     one [8 x 1026, 2560] tensor,        one [8 x 1026, 2560] linear,
     summed across the codebook axis     reshaped to [L, 8, 1026]
  |  8 codebooks x 1026 vocab, 25 fps, delay-patterned
  v
+------------------------------------------------------+
| Higgs Audio v2 Tokenizer  (separate checkpoint)       |
|  semantic branch: HuBERT, 12 layers, 768 dim, 16 kHz  |
|  acoustic branch: DAC, 1024-entry codebooks,          |
|                   hop 960, downsample x320            |
+------------------------------------------------------+
  v
24 kHz waveform
```

Figure 2 — the Higgs TTS 3 inference stack.

The tokenizer is the interesting part

Most codec LMs stack a purely acoustic codec (DAC, EnCodec) under the language model, leaving the LM to learn what the sounds mean on its own. Boson instead trained a unified tokenizer that fuses a semantic branch — a HuBERT encoder at 16 kHz, capturing phonetic content — with an acoustic branch — a DAC-style residual quantiser, capturing timbre and detail — into one token stream at 24 kHz.

This is plausibly why Khmer works at all despite being unlisted. A semantically-grounded tokenizer generalises to phonation the LM saw little of during training, because the phonetic structure is already factored into the token space by the tokenizer rather than having to be learned from scratch by the language model.

The delay pattern

Eight codebooks must be emitted per 40 ms frame, but an autoregressive model emits one position at a time. The standard solution — used here — staggers the codebooks so each step predicts one new codebook's token while the others lag behind:

```
codebook c is shifted by c steps
raw [T, 8]  ->  delayed [T + 7, 8]
padded with BOC = 1024 before its span and EOC = 1025 after
(hence vocab 1026 = 1024 codes + BOC + EOC)
```

At generation time a small state machine ramps the delay up, watches for EOC, and winds down; the rows are then de-delayed and handed to the codec. Any training code must apply exactly this transform to its targets — see §4.4.

Prompt format

```
<|tts|>  [ <|ref_text|>  ...reference transcript...  ]
         [ <|ref_audio|> ...one placeholder per ref audio token... ]
         <|text|>  ...the text to speak...
         <|audio|>   <- generation starts here
```

Audio positions are marked with placeholder id −100 in the text stream and overwritten with the fused audio embedding before the forward pass. The bracketed parts are the zero-shot voice-cloning path; omit them and the model uses its own default voice, which is how the evaluation ran it.

Inline control tokens

All tags use <|category:value|> syntax and can be inserted mid-utterance. VoxCPM2 has no equivalent.

Category	Values
Emotion (21)	elation, amusement, enthusiasm, determination, pride, contentment, affection, relief, contemplation, confusion, surprise, awe, longing, arousal, anger, fear, disgust, bitterness, sadness, shame, helplessness
Style (3)	singing, shouting, whispering
Sound effects (9)	cough, laughter, crying, screaming, burping, humming, sigh, sniff, sneeze — pair each with the matching onomatopoeia
Prosody	speed very_slow / slow / fast / very_fast (≈0.65×–1.4×); pause (400–700 ms); long_pause (700–1500 ms); pitch_low (−3 st); pitch_high (+2.5 st); expressive_high / expressive_low

Whether these transfer to Khmer is untested. They were trained on the documented languages; nothing establishes that an emotion tag produces Khmer-appropriate prosody rather than an English-shaped one. This is cheap to check by hand and worth checking before relying on it.

4.3 The training situation

Boson ship no training or fine-tuning code for Higgs TTS 3. This was verified rather than inferred:

- The official repository contains boson_multimodal/, examples/ and tech_blogs/. examples/ holds generation.py, serve_engine/, vllm/, voice_prompts/ and scene_prompts/ — **inference and serving only**. There is no trainer, no dataset loader, no config directory.
- The shipped remote-code class HiggsMultimodalQwen3ForConditionalGeneration implements generate_speech(), _build_prompt_ids(), _prefill_embeds() and a sampler state machine. **It has no forward() that accepts labels and returns a loss** — it is not wired for gradient descent.
- Nothing in the model card, the Boson blog post, or the LMSYS SGLang-Omni write-up discusses fine-tuning.
- Training data is undisclosed. Higgs Audio v2 was described as pretrained on a 10M-hour corpus ("AudioVerse"); no comparable figure is published for v3, and no per-language breakdown exists for either. There is no way to know how much Khmer, if any, the model saw.

The nearest prior art is the community LoRA trainer JimmyMa99/train-higgs-audio — for v2, not v3. v3 changed the architecture (v2's DualFFN is gone; v3 uses a plain Qwen3 backbone with a fused multi-codebook embedding and head), so that code is a useful reference for the data pipeline, not a drop-in.

4.4 What writing a trainer would involve

This is tractable — it is a standard codec-LM training loop, and the architecture is fully legible from the shipped remote code — but it is real engineering, not configuration.

#	Step	Detail
1	Encode audio to codes	Run bosonai/higgs-audio-v2-tokenizer over each clip to get [T, 8] integer codes at 25 fps.

2	Apply the delay pattern	apply_delay_pattern() already exists in the shipped remote code: [T, 8] → [T+7, 8], BOC-padded before each codebook's span, EOC after.
3	Build the sequence	Reuse _build_prompt_ids() and _prefill_embeds() **verbatim** so training and inference agree exactly. A mismatch here is silent and fatal.
4	Write the missing forward()	Embed via HiggsFusedMultiTextEmbedding, run the Qwen3 backbone, project with HiggsFusedMultiTextHead to [L, 8, 1026], cross-entropy against the next delayed frame across all 8 codebooks — masking BOC/EOC padding and the whole text span.
5	Attach LoRA	Via peft, on the backbone's attention and MLP projections. Full fine-tuning 4B params in bf16 needs ~60–80 GB with an 8-bit optimiser; LoRA brings it into single-24 GB range. The fused embedding and head are tied to the text embedding — decide deliberately whether to adapt them.
6	Validate by generation	Codec-LM loss is a poor guide to output quality. Synthesise a held-out set every N steps and listen.

Data requirements would resemble VoxCPM2's — paired 16 kHz-or-better audio with accurate Khmer transcripts, silence-trimmed, 3–30 s clips — and the same corpus scarcity applies. See §5.

5. Data requirements for Khmer

Both models are trained end-to-end on graphemes, so the data question reduces to one thing: paired (text, audio) at adequate quality. This section covers what that means concretely and what actually exists for Khmer. It applies to both models equally.

5.1 What TTS data consists of

- **Audio.** Single speaker per file, minimal background noise or music, consistent loudness. VoxCPM2 requires exactly 16 kHz; a codec-LM trainer would want 24 kHz or better and resample. Higher source rates are always preferable — you can downsample, not upsample.
- **Transcripts.** Orthographically correct text matching the audio exactly. Neither model needs forced alignment, phonemes or romanisation — raw Khmer text is the input.
- **Speaker labels** for the voice-cloning path (VoxCPM2's optional ref_audio field pairs two clips from the same speaker). Emotion and style metadata matter only for controllable synthesis.
- **Aggressive silence trimming.** Under 0.5 s of trailing silence per clip. See §3.4 — this is the single highest-value preprocessing step, and neglecting it produces runaway generation.

5.2 Scale, by goal

Goal	What you need	Typical scale
Single-speaker, single-language, from scratch (classic VITS)	Clean studio recordings from one speaker, consistent mic and room	~10–25 hours (LJSpeech, ~24 h, is the benchmark)
Adapting a foundation model to one Khmer voice	Clean clips from one speaker	**5–50 clips** (LoRA)
Adapting a foundation model for Khmer prosody or a domain	One or two speakers, matched to the target domain	**50–500 clips** (LoRA)
Large-scale Khmer customisation	Multi-speaker, QC'd	1000+ clips (full fine-tune)
Adding a language a model does not know	Broad, diverse coverage of the language	**500+ hours** — not applicable to VoxCPM2 + Khmer
Training a multilingual foundation model	Millions of hours plus rich metadata	1–10M+ hours (VoxCPM2: 2M+; Higgs Audio v2: ~10M)

5.3 Khmer corpora that actually exist

Source	Size	Licence	Assessment
OpenSLR SLR42	866 MB (male set)	CC BY-SA 4.0	Google-collected, manually QC'd. The closest thing to a standard Khmer TTS corpus. **OpenSLR publishes no hour count, speaker count or sample rate** — download and measure before planning around it. Mirrored on HF as deepdml/openslr42-khmer-

			tts.
Panhapich/khmer-tts-processed	1k–10k rows	—	Pre-segmented; check provenance before relying on it.
Panhapich/khmer-english-codeswitch-tts	1k–10k rows	CC BY 4.0	Matches a code-switched use case directly, **but is tagged synthetic / tts-generated** — it is TTS output, so training on it distils another model's errors, accent and artefacts into yours. Useful for text; treat the audio with suspicion.
KLEA	~3,000 words	—	Word-level, not sentences. Good for pronunciation reference, not TTS fine-tuning.
MMS-lab (Khmer portion)	~40 h (project average)	Research	New Testament readings, single speaker. What backs facebook/mms-tts-khm. Domain-narrow and single-voice.

The honest summary

There is no large, clean, permissively-licensed Khmer TTS corpus. SLR42 is the only well-attested one and it is a single-gender set of unpublished size. Beyond a few hours, you will be recording it or licensing it.

Given that VoxCPM2 already speaks Khmer, **recording 200–500 clean sentences from one or two good speakers and running LoRA is a far better use of effort than assembling a large corpus.** Corpus quality dominates corpus size at this scale, and avoiding synthetic (TTS-generated) audio matters more than adding hours.

6. How TTS is evaluated — and why it fails for Khmer

TTS quality is evaluated along two largely independent axes: intelligibility (did it say the right words?) and naturalness or similarity (does it sound good, and like the intended speaker?). Both human and automatic metrics exist for each. This section sets out the standard toolkit, then reports what happened when it was applied to Khmer.

6.1 Subjective (human-rated) metrics

Metric	What it measures	How it works
MOS — Mean Opinion Score	Overall perceived quality and naturalness	Listeners rate samples 1–5; scores averaged. The oldest and still most-cited TTS metric, but expensive, slow, and not comparable across studies with different listener pools.
CMOS — Comparative MOS	Relative quality between two systems	Listeners hear A/B pairs and rate preference and strength on a signed scale, e.g. −3 to +3.
SMOS — Similarity MOS	How much a cloned voice resembles the target speaker	Same 1–5 scale, but the question is "does this sound like the reference speaker", not "is this good speech".
A/B preference (Arena-style)	Head-to-head system ranking	Blind pairwise votes aggregated into an Elo rating, as Chatbot Arena does for LLMs. HuggingFace's TTS-Arena is the standard public leaderboard of this kind.

6.2 Objective (automatic) metrics

Metric	Measures	How it is computed
WER / CER	Intelligibility	Run a strong ASR model (commonly Whisper-large-v3) on the synthesised audio and diff the transcript against the input text. The single most-reported number in modern TTS papers. **CER is preferred over WER for non-whitespace-segmented languages** — Khmer, Thai, Chinese, Japanese — since word boundaries are themselves ambiguous.
SIM / SECS	Voice-cloning fidelity	Cosine similarity between speaker embeddings of reference and generated audio, typically from a WavLM-large speaker-verification model or ECAPA-TDNN.
UTMOS	Predicted naturalness	A network trained to predict human MOS from self-supervised speech features — a fast, free proxy that avoids running a

		listening study per experiment.
DNSMOS / NISQA	Predicted perceptual quality, including noise and distortion	Similar MOS-predictor models, originally built for speech enhancement, now reused as a no-reference TTS quality check. DNSMOS reports SIG (signal), BAK (background) and OVRL.
MCD	Acoustic distance to a reference recording	Mel-cepstral distortion. Used when a ground-truth recording of the same sentence exists; less applicable to zero-shot systems, which have no single correct reference.
RTF	Inference speed, not quality	Synthesis time ÷ duration of resulting audio. Below 1.0 is faster than real time. Determines viability for live and streaming use.

6.3 Standard benchmark suites

- **Seed-TTS-eval** (ByteDance) — the most widely adopted zero-shot benchmark. Reports WER (via Whisper-large-v3) and SIM (via WavLM-large) on English and Chinese, plus a "hard" subset of linguistically tricky sentences.
- **CV3-eval** — a Common Voice-derived multilingual set reporting WER/CER, SIM and DNSMOS. This is the main route by which multilingual coverage beyond EN/ZH gets benchmarked, and the suite VoxCPM2 cites for its 11- and 30-language comparisons, Khmer included.
- **TTS-Arena / TTS-Arena V2** (HuggingFace) — public blind-listening Elo leaderboard; the human-preference equivalent of Seed-TTS-eval.
- **InstructTTSEval and the MiniMax Multilingual Test** — newer suites for instruction following ("say this angrily", "whisper this") and broader multilingual robustness.

6.4 What the vendors report

Benchmark	VoxCPM2 (self-reported)	Higgs TTS 3 (self-reported)
Seed-TTS-eval, test-EN	WER 1.84%, SIM 75.3%	Not published in comparable form
Seed-TTS-eval, test-ZH	WER 0.97%, SIM 79.5%	Not published in comparable form
Multilingual	30-language internal set, 1.68% average error	"Single-digit WER/CER" on 102 languages, two tiers
Khmer	**2.05% CER** (internal 30-language set)	**Not benchmarked — Khmer is not on either language list**
Speed	RTF ~0.30 on RTX 4090; ~0.13 with Nano-vLLM	Optimised for SGLang-Omni serving

Table 3 — vendor-reported figures. Both columns are self-reported and should be read as upper bounds. VoxCPM2's Khmer figure comes from a competitor comparison published by OpenBMB, not an independent third party; it is directionally credible given the size of the gap it claims over Fish Audio S2-Pro (75.15% CER on the same test), but it has not been independently reproduced.

6.5 First pass: the run that did not work

All four candidate models were run over the fixed 100-sentence Khmer set — 400 clips, zero failures — on an RTX 3060 and scored on CER (Whisper-large-v3, language=km), UTMOS, DNSMOS and RTF. The result was a table whose metric columns and listening verdict disagreed completely.

Model	CER median	UTMOS	DNSMOS OVRL	P.808	RTF median	Listening verdict
mms	98.8%	2.95	3.23	3.71	0.010	**Eliminated** — prosody
voxcpm2	100.7%	2.49	2.98	3.50	1.382	**Contender**
fish-s2	101.0%	3.75	3.21	3.79	2.651	**Eliminated** — correctness
higgs3	98.2%	2.98	3.14	3.84	0.745	**Contender**

Table 4 — the first pass, scored with Whisper-large-v3. Superseded for CER by Table 5; kept because the failure is instructive and because UTMOS, DNSMOS and RTF were never affected by it.

Why the CER column here is meaningless

Whisper-large-v3 cannot transcribe Khmer — it collapses into repetition loops. Median CER is ≈100% for every model, and 34 of 100 sentences drew a byte-identical transcript from two or more different models' audio, which is only possible if the transcripts describe Whisper rather than the audio. Decoder settings (temperature fallback, n-gram repetition blocking, language auto-detect) were each tried and ruled out as the cause.

6.6 Second pass: CER with a Khmer-capable ASR

The blocker was never CER as a metric — it was the absence of an ASR that can read Khmer. One now exists in house: a fine-tuned CTC model built on Meta's Omnilingual ASR 300M encoder with a grapheme-cluster vocabulary and a self-conditioned CTC head, published as Darayut/Omnilingual-ASR-Khm. Its reported accuracy on held-out natural speech is 2.24% CER in domain, 12.03% on FLEURS and 14.82% on OpenSLR SLR42 — against Whisper's effective 100%. Re-scoring the same 400 clips with it, using the harness's unchanged CER definition, produces a table that finally means something.

Model	CER median	CER mean	pure_khmer	code_switched	Listening verdict
voxcpm2	2.47%	4.48%	1.90%	3.19%	**Contender**
higgs3	8.28%	9.88%	8.90%	7.13%	**Contender**
mms	25.12%	24.84%	19.73%	31.20%	**Eliminated** — prosody
fish-s2	78.01%	77.81%	81.67%	73.76%	**Eliminated** — correctness

Table 5 — CER re-scored with a Khmer-capable ASR, same 400 clips, same CER definition (evaluation/metrics/khmer_text.py). Rows ordered by CER. The metric now agrees with the listening verdict instead of contradicting it.

The scorer independently reproduces two published benchmark figures

This is the strongest available evidence that the numbers above are real rather than an artefact of the scorer. OpenBMB's own 30-language benchmark reports **2.05% CER for VoxCPM2** on Khmer and **75.15% for Fish Audio S2-Pro**. Measured here — different test set, different ASR, different hardware — VoxCPM2 lands at 2.47% median and S2-Pro at 78.01%. Two independent reproductions, both close.

That also settles the question OpenBMB's figures previously left open. Their Khmer numbers were a vendor's self-report in a competitor comparison; §6.4 rated them directionally credible but unreproduced. They are now reproduced.

The scorer is not neutral, and the bias favours VoxCPM2

The ASR's training pool includes roughly **47 hours of VoxCPM2-synthesized Khmer** (19,825 rows from an LLM-authored corpus, 14,431 from VoxCPM2 voice cloning) and **no Higgs TTS 3 at all**. It has heard VoxCPM2's exact acoustic signature at length. Some of the gap between the two contenders is therefore domain familiarity, not synthesis quality. No VoxCPM2-free checkpoint exists — every saved checkpoint trained on a manifest containing the synth shards.

Two checks argue the effect does not explain the result. First, the calibration above: if the ASR were inflating VoxCPM2, VoxCPM2 would beat its published 2.05%, and it does not. Second, the VoxCPM2 synthetic data was **code-switch** audio, so familiarity should help most on the code-switched half — but VoxCPM2's head-to-head win rate is **higher on pure Khmer (39/50) than on code-switched (34/50)**, the opposite of the predicted pattern.

Read it as: the direction is trustworthy, the exact size of the gap is not. Treat “VoxCPM2 is several times more intelligible than Higgs TTS 3 in Khmer” as supported, and the specific ratio as an upper bound.

Head to head over the 100 sentences, VoxCPM2 records the lower CER on **73**, Higgs TTS 3 on **15**, with 12 ties — a broad and consistent advantage rather than one driven by a few outliers.

Higgs TTS 3's errors have a characteristic shape: on pure-Khmer sentences the ASR transcribes fragments of Latin script (“b”, “s troop”, “bad”) where Khmer belongs, i.e. it is hearing phonation that is not quite Khmer. That signature is modest in aggregate — 11 of 50 pure-Khmer utterances for Higgs against 8 for VoxCPM2 — so it illustrates the failure rather than carrying the argument; fish-s2, by contrast, shows it on 48 of 50.

6.7 Naturalness, re-examined

CER settles intelligibility. It does not settle naturalness, which is where the predictors and the listener disagreed in the first place: UTMOS put Higgs TTS 3 (2.98) above VoxCPM2 (2.49), and a native Khmer speaker hears the reverse. Before accepting “the predictor is deaf to Khmer”, the mundane explanations were tested.

The clips are not delivered at the same level: VoxCPM2 sits at −15.6 dBFS RMS and peaks at −0.2 dBFS, Higgs TTS 3 at −24.5 dBFS RMS peaking at −6.8 dBFS — a ~9 dB gap, and both UTMOS and DNSMOS respond to level and to silence padding. So every clip was re-scored under four conditions.

Condition	What it does	VoxCPM2 UTMOS	Higgs 3 UTMOS
raw	as synthesized (reproduces the first pass)	2.462	3.019
peak	peak-normalized to −1 dBFS	2.459	2.954
loudness	BS.1770 loudness-normalized	2.489	3.003

	to −23 LUFS		
trimmed	loudness-normalized, then silence trimmed	2.468	2.941

Table 6 — UTMOS medians under level and silence control. The ranking does not move under any condition. This is a negative result, and a useful one: it rules out the cheap fix.

Level accounts for a sliver of the gap and no more. Per utterance, VoxCPM2 takes the higher UTMOS on 18 of 100 sentences as synthesized; equalizing peak level lifts that to 26, loudness-matching to 21, and trimming silence to 22. The mean margin stays near −0.46 throughout. Something real is being measured — it is simply not what a Khmer listener is judging.

> **UTMOS is not merely uninformative about Khmer — it is inverted**
>
> Across all 400 clips, the rank correlation between UTMOS and CER is **rho = +0.55**. The sign is the finding. If UTMOS tracked whether the audio says the Khmer text, the correlation would be strongly negative — better naturalness, fewer errors. Positive means the opposite: **the clips UTMOS likes best are the ones that get the Khmer most wrong.**
>
> At model level the inversion is nearly total. Ordered by CER the ranking is voxcpm2, higgs3, mms, fish-s2; ordered by UTMOS it is very close to the reverse — fish-s2 has the worst CER in the run (78.01%) and the best UTMOS (3.88), while VoxCPM2 has the best CER (2.47%) and the worst UTMOS (2.46).
>
> Within a single model's own output the correlation is near zero or weakly negative, as it should be. The inversion is entirely a **between-model** effect, which is exactly the comparison the metric was being used to make.

The mechanism is straightforward once stated. UTMOS was trained on MOS studies of English and Japanese speech and rates acoustic smoothness and prosodic plausibility against those languages. It has no way to know whether a Khmer sentence was pronounced correctly, and Higgs TTS 3's slightly non-Khmer phonation — the same thing the ASR transcribes as stray Latin — is not a defect on that scale. A model can therefore score well by sounding clean while saying the wrong thing, which is precisely what fish-s2 does in the extreme and what separates the two contenders in miniature.

One bias-free check corroborates this without any learned model. A pure-arithmetic speaking-rate guard — flag any utterance delivered far faster than the model's own median, which is what truncation looks like — flags **8 of 100** fish-s2 utterances and **zero** for either contender. It catches the failure UTMOS rewarded, costs nothing, and cannot be biased toward any model.

6.8 So how do you measure naturalness?

Intelligibility is now settled by CER. Naturalness is not, and the honest answer is that no automatic metric can settle it for Khmer today. Two candidate shortcuts were tested and both failed, which is worth recording so they are not tried again.

The two shortcuts that do not work

- **MOS predictors (UTMOS, DNSMOS).** Inverted for this comparison — §6.7. Not usable for ranking at any level of post-processing; level and silence were controlled and the ranking held.

- **Prosody statistics as a naturalness proxy.** Pitch variation is the classic correlate of expressive versus flat delivery, so F0 statistics were measured over all 400 clips. The measure was given a falsifiable test: MMS was eliminated by ear for flat, robotic prosody, so it should show the least pitch movement. **It shows the most** — an F0 standard deviation of 5.66 semitones against 3.06 for VoxCPM2 and 3.35 for Higgs TTS 3. Wide but wrongly-placed pitch movement reads as robotic, not expressive, and the statistic cannot tell the difference. The proxy is rejected on its own test.

> **A confound to fix before running any listening test**
>
> The two contenders are not speaking in the same voice. Measured median F0 is about **206 Hz for VoxCPM2** against **114 Hz for Higgs TTS 3** — close to an octave apart, so a different apparent speaker and plausibly a different apparent gender. Ask a listener which sounds more natural and part of the answer is which voice they prefer, which is a property of an arbitrary default rather than of the synthesis.
>
> Both models do zero-shot voice cloning, so the fix is available: re-synthesize both from the same reference clip and compare those. It costs one re-run. Without it, a decisive result (say 70/30) is still informative — timbre preference is unlikely to be worth that much across five listeners — but a narrow one cannot be separated from voice preference.

The instrument that does work

A blind, randomized, forced-choice listening test. For naturalness this is not a fallback for want of a metric — it is the definition of the quantity. What makes it evidence rather than an impression is the design:

Property	What it means	Why it is not optional
Blind	Model identity never shown to the listener	Knowing which is which is enough to produce the expected answer.
Randomized	Left/right assignment per trial, order per listener	Listeners favour the first option heard; unrandomized, that bias silently attaches to whichever model was placed first.
Forced choice	2AFC — pick one, or declare a tie	Needs no scale calibration between listeners, and resolves a given difference in far fewer trials than rating each system separately.
Anchored	A share of trials pit a contender against a known-bad model	An unsupervised remote test cannot otherwise distinguish a careful listener from someone clicking through. Six anchors minimum: at four, random clicking passes 31% of the time.
Single question	Naturalness only, with explicit instruction to ignore word errors	CER already answers intelligibility. Without the instruction the test re-measures it — where VoxCPM2 already wins — and manufactures agreement instead of testing for it.

Table 7 — the design requirements. Implemented in evaluation/listening_test.py, which emits a single self-contained HTML file per study and a separate answer key that never travels with it.

How many listeners, and how many sentences

This is the part most often got wrong, and getting it wrong produces a null result that reads like a finding. Against a 50% null at 95% confidence and 80% power:

If the true preference is…	Trials needed	Practical configuration
70 / 30	47	2 listeners × 25 sentences
65 / 35	85	3 listeners × 30 sentences
60 / 40	194	**5 listeners × 40 sentences** — the default
55 / 45	776	10 listeners × 80 — rarely worth it; at this margin the two systems are interchangeable for most purposes

Table 8 — statistical power for a two-alternative forced choice. Trials from one listener are correlated, so the totals here flatter the real power; the honest unit of replication is the listener, which is why the analysis reports per-listener rates and a sign test across listeners alongside the pooled interval.

The reporting rule that follows: a preference rate is not a result without an interval. “VoxCPM2 preferred 58% of the time” says nothing; “58%, 95% CI [50.4%, 65.5%]” says the interval barely excludes chance, and a reader can see that one more ambivalent listener would sink it. evaluation/listening_analyse.py reports the Wilson interval, an exact binomial p-value, per-listener rates, catch-trial pass rates, and a side-bias check — and refuses to call a winner when the interval includes 50%.

One further caution, visible in a dry run of the analysis: five listeners can produce a pooled interval that excludes 50% while the listener-level sign test does not, because 200 trials from five people are not 200 independent observations. When the two disagree, the sign test is the conservative reading and the study is better described as suggestive.

6.9 The evaluation plan this leaves

In priority order:

- **CER against a Khmer-capable ASR — now the primary metric.** It works, it is cheap (400 clips in ~25 seconds), and it agrees with trained ears. Quote it with the scorer named and its bias stated, exactly as §6.6 does.
- **Blind A/B preference with multiple Khmer-speaking listeners.** Still the only trustworthy naturalness measure. Randomise order, hide model identity, aggregate pairwise votes into a preference rate with confidence intervals. This is what would separate the two contenders on naturalness, which CER does not address.
- **Speaking-rate and duration guards.** Free, ASR-free, unbiasable, and they catch the truncation failure mode outright.
- **RTF and VRAM on the target hardware.** Unambiguous and directly decision-relevant.
- **UTMOS and DNSMOS — as artefact detectors only.** They are useful for spotting buzz, clicks and distortion within one model's output. Do not rank models with them for Khmer: at that job they are inverted.

7. Comparison and recommendation

	VoxCPM2	Higgs TTS 3
Parameters	2B	4B
Audio representation	Continuous latents (tokenizer-free)	8 discrete codebooks @ 25 fps
Backbone	MiniCPM-4	Qwen3-4B
Output sample rate	**48 kHz**	24 kHz
Khmer support	**Documented** — 1 of 30 languages, 2.05% CER claimed	**Undocumented** — works anyway, no vendor commitment
Licence	**Apache-2.0**	Research / non-commercial; Creator Use Grant with attribution
Official fine-tuning	**Yes** — guide, YAML configs, LoRA, data validator	**None** — you would write the trainer
Effort to a first adapted result	**A YAML file and a 24 GB card**	Weeks of implementation, then a 24 GB card
Expressive control	Natural-language instruction, prepended as **(instruction)text**	**21 emotions, 3 styles, 9 sound effects, prosody tags**, inline
Streaming synthesis	**generate_streaming()** yields chunks	Not in the shipped API
Measured RTF (RTX 3060)	1.382	**0.745**
Measured CER (Khmer ASR)	**2.47% median**	8.28% median
Head-to-head CER wins	**73 / 100**	15 / 100 (12 ties)
Measured UTMOS	2.46 (last of four)	3.02 (second of four)
Listening verdict	**Contender** — preferred by ear	**Contender**

Table 12 — side by side. The CER row is the one that changed: measured against a Khmer-capable ASR it separates the two contenders, where every metric in the first pass failed to. The UTMOS row is retained only to show what was measured — per §6.7 it is inverted for Khmer and must not be used to rank.

7.1 Recommendation

Build on VoxCPM2. Keep Higgs TTS 3 as the baseline to beat.

Three independent lines now point the same way. **Licensing:** one of them can be shipped and the other cannot. **Effort:** one takes a config file to improve, the other takes a from-scratch trainer. **Measured quality:** VoxCPM2 is several times more intelligible in Khmer — 2.47% median CER against 8.28%, the lower error on 73 of 100 sentences — which corroborates the listening verdict rather than contradicting it.

The earlier draft of this review declined to name a winner between the two contenders, because at that point no valid metric separated them and only one listener had judged. That has changed on the metric side. **VoxCPM2 is the recommendation.** What is still open is naturalness specifically — CER measures whether the words are right, not whether the delivery is pleasant — and that still wants the blind multi-listener test in §6.8.

Where Higgs TTS 3 nonetheless earns its place:

- **As the baseline to beat.** It is the strongest zero-shot Khmer output in this survey, obtained with no adaptation at all. Any VoxCPM2 fine-tune should be A/B'd against it; if the fine-tune does not clearly win, the fine-tune is not done.
- **On speed.** Nearly 2× faster than VoxCPM2 on the same card. For a latency-bound application that is a real argument.
- **For expressive and creator work.** The control-token vocabulary has no VoxCPM2 equivalent, and the Creator Use Grant covers monetised content with attribution.
- **As the fallback.** Should VoxCPM2 fine-tuning fail to improve on the base model, writing a Higgs trainer becomes the reasonable next investment — rather than the first one.

If Higgs TTS 3 is pursued, the first step is not code. It is confirming with Boson that a commercial licence is obtainable on acceptable terms, since without one the work cannot be deployed regardless of how well it turns out.

7.2 Proposed path for a Khmer voice

#	Step	Detail
1	Record or license 200–500 clean Khmer sentences	One or two speakers, 16 kHz, 3–15 s per clip, trailing silence trimmed under 0.5 s. Prefer recording over scraping — corpus quality dominates everything downstream. Avoid synthetic (TTS-generated) audio entirely.
2	Hold out ~10% as a validation split	Run `voxcpm validate` on both splits before committing GPU time.
3	LoRA fine-tune	r=64, alpha=64, lr 1e-4, 1000 steps, enable_lm and enable_dit both true, on a rented 24 GB card. Save every 500 steps.
4	Listen to every checkpoint	Do not select on loss. Overfitting arrives within a few hundred steps on small datasets.
5	Re-run the fixed evaluation set	Synthesise the same 100 sentences and A/B the LoRA output against both the VoxCPM2 base clips and the Higgs TTS 3 clips.
6	Judge by blind listening	Per §6.6 — multi-listener A/B plus a transcription task. No automatic metric in this project can rank Khmer TTS.

8. Using them: the practical surface

Sections 3 and 4 cover what these models are. This section covers what you can actually ask them to do. Everything below is read from the shipped code — the voxcpm 2.0.3 package installed in this repo, and the modeling_higgs_multimodal_qwen3.py remote code that ships with the Higgs weights — rather than from either vendor's feature list.

8.1 Capability matrix

Capability	VoxCPM2	Higgs TTS 3
Plain synthesis	`generate(text)`	`generate_speech(text, tokenizer)`
Streaming output	**Yes** — `generate_streaming()` yields chunks as they are produced	Not in the shipped API; serving stacks (vLLM, SGLang-Omni) handle it outside the model class
Zero-shot voice cloning	`reference_wav_path` — isolated via ref_audio tokens	`reference_audio` + `reference_sample_rate`, optionally `reference_text`
Continuation / in-context cloning	`prompt_wav_path` + `prompt_text` — the model continues the prompt's voice and manner	Same idea via `reference_text` + `reference_audio`; no separate continuation mode
Pre-encoded / cached voice	Re-encodes the reference on every call	**`reference_codes`** — encode a voice once, reuse the codes. Meaningful saving if one voice serves many requests
Expressive control	Natural-language instruction (§8.2)	Fixed inline tag vocabulary (§8.2)
Sampling controls	`cfg_value` 1.0–3.0 (guidance strength)	`temperature`, `top_p`, `top_k` — the usual LM sampling dials
Quality / speed dial	**`inference_timesteps` 4–30** — fewer diffusion steps is faster and rougher	`max_new_tokens` only; no quality dial
Reference denoising	**Built in** — `denoise=True` runs ZipEnhancer over the reference first	None; clean your reference yourself
Runaway / truncation guard	**Built in** — `retry_badcase` re-rolls when the audio-to-text ratio exceeds a threshold (default 6.0)	None
Text normalization	`normalize=True`	None
Batch CLI	`voxcpm batch --input lines.txt --output-dir out/`	Example scripts only
Fine-tuning	**LoRA, hot-swappable at runtime** — `load_lora()`, `set_lora_enabled(False)`, `unload_lora()`	No training code ships at all
Dataset validator	**`voxcpm validate --manifest …`**	None

Table 10 — the usage surface of each model, from the shipped code. The pattern is consistent: VoxCPM2 ships more operational machinery (streaming, denoising, retry, validation, adapters), Higgs ships a richer expressive vocabulary and a cached-voice path.

8.2 Expressive control — two different paradigms

Both models can be told how to say something, but they take opposite approaches, and the difference matters more than the feature counts suggest.

VoxCPM2 — open-ended, natural language

There is no tag vocabulary. You write an instruction in plain language and it is prepended to the text in parentheses; the model was trained to read a leading parenthesised descriptor as a voice and style instruction. The CLI exposes it as --control, but the mechanism is only string concatenation, so the plain Python API gets it for free:

```
# what the CLI does internally is exactly this:
#     final_text = f"({control}){text}"  if control else text

voxcpm design --text "Hello world" --control "warm female voice" --output out.wav

# and therefore, from Python, with no special argument:
wav = model.generate(text="(warm female voice, speaking slowly)Hello world")
```

- **Unbounded.** “an elderly man, tired, speaking gently”, “a news anchor, brisk and formal”, “whispering, conspiratorial” — anything you can describe. It is not limited to a list somebody chose in advance.
- **Whole-utterance only.** The instruction sits at the front and colours the entire utterance. There is no way to change emotion halfway through a sentence.
- **Unverifiable.** Nothing validates the instruction. A descriptor the model does not understand is silently ignored, or worse, partly spoken. You find out by listening.
- **Mutually exclusive with continuation mode.** The CLI rejects --control together with --prompt-text: you are either designing a voice from a description or continuing one from audio, not both. It does combine with --reference-audio.

Higgs TTS 3 — a closed, composable tag set

A fixed vocabulary using <|category:value|> syntax, insertable anywhere in the text, including mid-sentence:

```
text = "<|emotion:sadness|> I have to tell you something. "
       "<|long_pause|> <|emotion:relief|> But it turned out fine. "
       "<|laughter|> haha"
```

Category	Count	Values
Emotion	21	elation, amusement, enthusiasm, determination, pride, contentment, affection, relief, contemplation, confusion, surprise, awe, longing, arousal, anger, fear, disgust, bitterness, sadness, shame, helplessness
Style	3	singing, shouting, whispering
Sound effects	9	cough, laughter, crying, screaming, burping, humming, sigh, sniff, sneeze — each paired with the matching onomatopoeia
Prosody	—	speed very_slow / slow / fast / very_fast (≈0.65×–1.4×); pause (400–700 ms);

		long_pause (700–1500 ms); pitch_low (−3 st); pitch_high (+2.5 st); expressive_high / expressive_low

Table 11 — the full Higgs TTS 3 control vocabulary. Composable, positional, and checkable — an unknown tag is a visible mistake rather than a silent one.

> **Neither model's expressive control is verified for Khmer**
>
> Both mechanisms were trained on each vendor's documented languages. Khmer is one of VoxCPM2's 30, but its control instructions are written in English, and nothing establishes that an English descriptor steers Khmer prosody the way it steers English prosody. For Higgs the gap is wider: Khmer is not on its language list at all, so its tags are unverified in a language the model was never documented to speak.
>
> This is a cheap thing to settle and nobody has settled it. Synthesize ten Khmer sentences under three or four emotion settings, listen, and you will know. Do that before designing any product feature on top of either mechanism.

8.3 The same job, written both ways

Basic synthesis, then the same sentence in a cloned voice:

```
# ---------- VoxCPM2 ----------
from voxcpm import VoxCPM

model = VoxCPM.from_pretrained("openbmb/VoxCPM2")           # + lora_weights_path=...
wav = model.generate(text="<Khmer text>")                    # 48 kHz float32 numpy

wav = model.generate(
    text="<Khmer text>",
    reference_wav_path="speaker.wav",   # zero-shot clone
    denoise=True,                       # clean the reference first
    cfg_value=2.0,                      # 1.0-3.0, guidance strength
    inference_timesteps=10,             # 4-30, quality vs speed
    normalize=True,
)

for chunk in model.generate_streaming(text="<Khmer text>"):  # play as it arrives
    play(chunk)

# ---------- Higgs TTS 3 ----------
wav = model.generate_speech(text, tokenizer)                 # 24 kHz float32 torch

wav = model.generate_speech(
    text, tokenizer,
    reference_audio=ref_tensor,         # or reference_codes=cached, encoded once
    reference_sample_rate=24000,
    reference_text="<transcript of the reference>",   # improves cloning
    temperature=1.0, top_p=0.95,
    max_new_tokens=2048,
)
```

8.4 What this means for a Khmer product

- **If you need audio to start playing before it is finished,** VoxCPM2 is the one with a streaming generator in the box. This partly offsets its worse RTF: 1.38 real-time factor matters much less when the first chunk arrives quickly than when the whole file must finish first.

- **If you serve one or two fixed voices,** Higgs's `reference_codes` lets you encode each voice once and skip re-encoding on every request. VoxCPM2 has no equivalent cache and re-reads the reference each call.
- **If you need reliable, repeatable delivery,** Higgs's closed tag set is the safer instrument — a typo is a visible error, and the same tag means the same thing every time. VoxCPM2's free-text control is more expressive in principle and less predictable in practice.
- **If robustness matters more than range,** VoxCPM2's built-in bad-case retry deserves attention: it re-rolls generation when the audio-to-text ratio goes out of bounds, which is precisely the truncation failure that made fish-s2 unusable and that §6.7 had to catch with a separate guard. Higgs has no such protection.

9. Sources

VoxCPM2

- Zhou et al., “VoxCPM: Tokenizer-Free TTS for Context-Aware Speech Generation and True-to-Life Voice Cloning”, arXiv:2509.24650
- openbmb/VoxCPM2 model card, Hugging Face — language list, 2M-hour claim, 48 kHz output
- OpenBMB/VoxCPM on GitHub — benchmarks and install instructions
- VoxCPM Fine-Tuning Guide (voxcpm.readthedocs.io) — commands, YAML, VRAM, manifest schema
- VoxCPM Fine-Tuning FAQ (voxcpm.readthedocs.io) — data volumes, forgetting, failure modes
- config.json from the openbmb/VoxCPM2 checkpoint — every architecture figure in §3.1–3.2
- voxcpm 2.0.3 package source: training/validate.py, training/data.py, modules/layers/lora.py, cli.py

Higgs TTS 3

- bosonai/higgs-tts-3-4b model card, Hugging Face — architecture table, 102-language tiers, control tokens, licence and Creator Use Grant
- Boson AI — “Higgs TTS 3” blog post (boson.ai/blog/higgs-audio-v3-tts)
- LMSYS — “Higgs Audio v3 TTS on SGLang-Omni” (lmsys.org, 4 June 2026)
- boson-ai/higgs-audio on GitHub — repository contents, verified to contain no training code
- config.json from multimodalart/higgs-audio-v3-tts-4b-transformers — every backbone figure in §4.2
- config.json from bosonai/higgs-audio-v2-tokenizer — semantic and acoustic branch specifications
- modeling_higgs_multimodal_qwen3.py — delay pattern, BOC/EOC ids, prompt format, fused embedding and head, and the absence of a loss-returning forward()
- JimmyMa99/train-higgs-audio on GitHub — community LoRA trainer for Higgs Audio v2 (not v3)

Data and evaluation

- Pratap et al., “Scaling Speech Technology to 1,000+ Languages” (MMS paper), JMLR 2024, arXiv:2305.13516
- OpenSLR SLR42 — High quality TTS data for Khmer (openslr.org/42)
- seanghay/KLEA — Khmer word-to-speech model
- seanghay/khmertagger — inverse text normalisation and segmentation for Khmer
- Ito & Johnson — the LJSpeech dataset
- Seed-TTS-eval overview, EvalScope documentation
- Hugging Face TTS-Arena (huggingface.co/spaces/TTS-AGI/TTS-Arena)
- Artificial Analysis — Text-to-Speech benchmarking methodology
- Zilliz — standard evaluation metrics for TTS quality

In-house evidence

- Darayut/Omnilingual-ASR-Khm on Hugging Face — the Khmer CTC ASR used for the §6.6 re-scoring; Meta omniASR_CTC_300M encoder, grapheme-cluster vocabulary, self-conditioned CTC head
- evaluation/score_cer_khmer.py — the re-scoring CLI, including the scorer-bias analysis
- evaluation/score_naturalness.py — UTMOS and DNSMOS under level and silence control
- evaluation/audio_stats.py — ASR-free signal diagnostics and the truncation guard
- evaluation/results/cer_khmer_asr.json, naturalness.json, audio_stats.json — the numbers behind Tables 5-7, read directly by this document's build script

- eval-set/eval.json — the fixed 100-sentence Khmer test set (50 pure Khmer, 50 code-switched)
- evaluation/results_report.md and evaluation/results/*/scores.json — the full four-model, 400-clip synthesis and scoring run, including raw Whisper transcripts
- docs/09-voxcpm2-architecture-and-training.md and docs/10-higgs-tts-3-architecture-and-training.md — the working notes this review consolidates

Prepared for Smean AI · ស្មៀន · Built by Cambodians, for every Khmer speaker.